

Project Title: Surface Crack Detection Using Different Variants of Convolutional Neural Network

Title: Surface Crack Detection Using LeNet and PySpark

Team Members: Ganesh Vannam, Nandhika Rajmanikandan, Muneendra Magani and Madhumitha Mandayam.

Problem Statement:

The primary goal of this project is to detect surface cracks in images using convolutional neural networks (CNNs). Detecting surface cracks is crucial in various industries, such as manufacturing and construction, as undetected cracks can lead to structural failures. The project aims to:

- Classify images as either cracked or non-cracked.
- Implement a LeNet CNN model to efficiently process and classify image data.
- Leverage PySpark to handle large-scale data processing across multiple VMs.

Potential Challenges:

This project involves specific challenges, including:

- **Model Training:** Ensuring that the LeNet model converges well with the given data, while managing memory constraints and avoiding overfitting.
- **Data Augmentation:** Since the dataset is relatively small, data augmentation was employed to improve generalization and avoid overfitting

Implementation Plan:

CNN Model Design:

This project will deploy a LeNet model with the following structure:

- **Convolutional Layers:** Two convolutional layers with activation functions will be used to extract features from the input images, beginning with a basic architecture to evaluate effectiveness.
- **Pooling Layers:** Each convolutional layer will be followed by max-pooling layers, reducing dimensionality while preserving key features.
- **Fully Connected Layers:** Dense layers will then aggregate these extracted features for final classification.
- **Output Layer:** A softmax activation function will be used to classify images into specific glitch categories.

Hyperparameters:

- **Batch Size:** Starting with a default of 64, we may adjust this based on VM memory constraints.

- **Learning Rate:** Initial learning rate of 0.002; fine-tuning will be performed using grid search.
- **Number of Epochs:** Set to 10 as a default; early stopping may be applied to avoid overfitting.
- **Optimizer:** Adam optimizer, known for adaptive learning rates.

Hyperparameter Tuning:

Grid search or Bayesian optimization will be applied to adjust the learning rate, batch size, and number of filters. Early stopping and learning rate scheduling will help identify the optimal number of epochs for best results.

Techniques to Prevent Overfitting:

- **Data Augmentation:** Random transformations, including rotations, shifts, and flips, will enhance the diversity of the training data.
- **Dropout Layers:** Dropout layers will be included to randomly deactivate units during training to reduce overfitting.
- **Regularization:** L2 regularization will be used to penalize large weights, further helping to prevent overfitting.
- **Cross-Validation:** k-fold cross-validation will be conducted to validate model robustness and mitigate overfitting.

Dataset: The dataset used for this project was the **Surface Crack Detection dataset** from Kaggle, containing two main categories:

- **Positive** (Cracked surfaces)
- **Negative** (Non-cracked surfaces)

Link: <https://www.kaggle.com/datasets/arunrk7/surface-crack-detection?resource=download>

Spark Integration with Multiple VMs:

To handle big data, the CNN model will be implemented using Spark's MLlib on a distributed system with multiple VMs. Each VM will process a subset of the data, enabling parallel processing to speed up training.

Data Splitting and Performance Evaluation:

- **Data Splitting:** The dataset will be divided into training (70%), validation (15%), and testing (15%) sets.
- **Performance Metrics:** Model evaluation will include metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis to assess classification performance. Precision will be prioritized to monitor false positives, as it measures the ratio of true positives among all positive predictions. A higher precision indicates fewer false positives.

Image Data Augmentation:

Applying data augmentation techniques such as horizontal and vertical flipping, rotation, and adding noise to enhance the training dataset. These augmentations should be incorporated into the PySpark pipeline to maintain scalability and ensure efficient distributed processing.

Conclusion:

This project demonstrated the ability of the LeNet CNN architecture, integrated with PySpark for distributed processing, to classify surface cracks in images. The model successfully detected surface cracks with high accuracy and precision, reducing false positives. This approach holds promise for real-time crack detection in industrial applications, contributing to safety and maintenance efforts.

Note: The remaining team members will submit their individual proposals and methodologies for their respective parts of the project.